第4章 8086寻址方式与指令系统

西南交通大学计算机与人工智能学院

keli@home.swjtu.edu.cn

本章主要内容

1 概述

2 8086寻址方式

3 8086指令系统

4.1 概述

- 通常一条指令包括两部分：
  1. 操作码:决定要完成的操作
  2. 操作数:指参加运算的数据或是该数所在的内存单元的地址。
- 指令的一般格式如下：
  1. 没有操作数的指令称为无操作数指令。
  2. 有两个操作数的指令称为双操作数或二地址指令。

操作码	操作数1	操作数2	……	操作数n

- 无操作数格式：

  例：CLI ；　功能：清IF位

- 单操作数格式：

  例：INC AX ；功能：（AX）+1→AX

- 双操作数格式：

  例：MOV ES，BX ；功能（BX）→ ES

  （ES是目标操作数，BX是源操作数。）

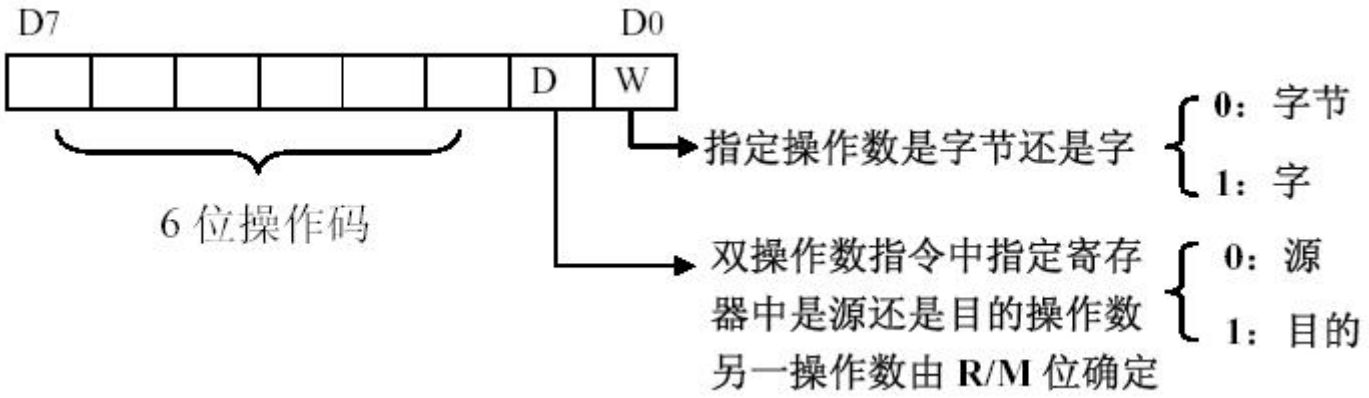
  例：ADD AX，[BX]；功能：（DS：（BX））+（AX）→ AX

  （AX是目标操作数，[BX]是源操作数。）

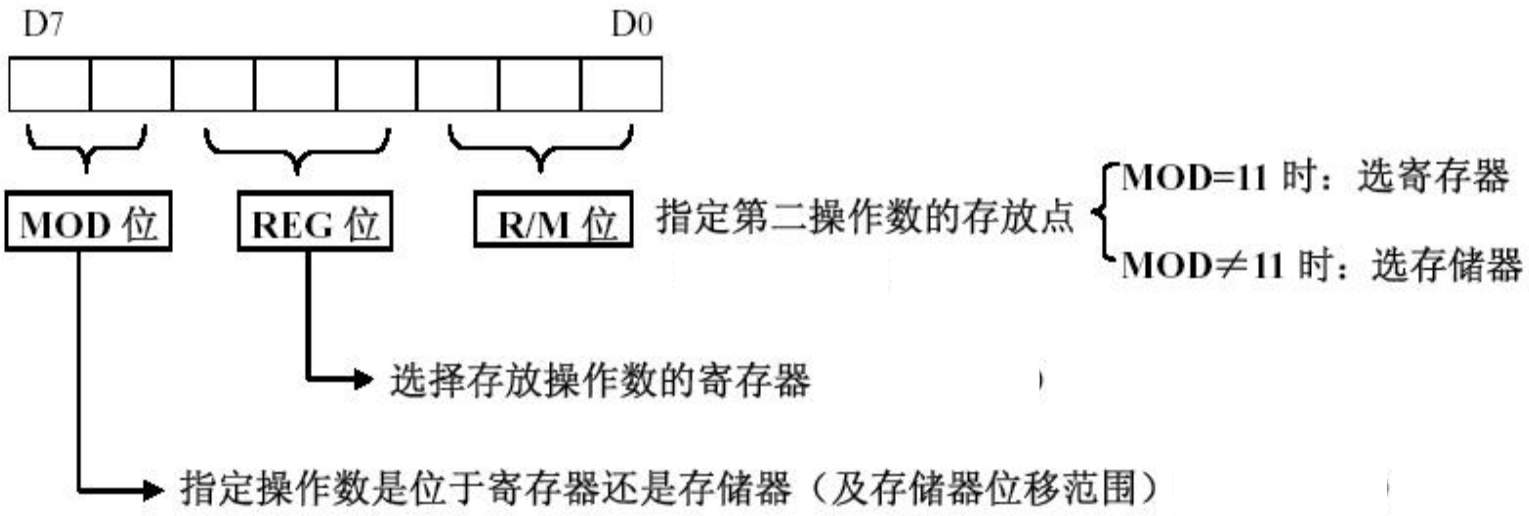
1 . 8086机器码的指令结构

8086/8088 CPU 的指令格式采用定长操作码，变长指令码。

① 指令首字节：为操作码，其结构为：



② 指令第二字节：指示操作数的存放地及寻址方式



③ 指令第三、四字节： 8/16位立即数或8/16位地址偏移量

④ 指令第五、六字节：指令中既有立即数又有地址偏移量

2. 寻址技术

① 操作数地址表示多样化需要。

② 压缩操作数地址字段的长度。

寻址技术是通过数据寻址方式的设定，压缩指令长度，并且灵活的寻找操作数地址。

指令中的形式地址A $\xrightarrow{\text{寻址方式}}$ 有效地址EA(段内偏移地址)

- 寻址方式：指令中给出的求出操作数有效地址的方法。
- 寻址操作：计算机按照指令给出的寻址方式求出操作数有效地址的过程。

4.2 8086寻址方式

8086的七种基本的数据寻址方式

(1)立即寻址；
(2)寄存器寻址；
(3)直接寻址；
(4)寄存器间接寻址；
(5)寄存器相对寻址；
(6)基址变址寻址；
(7)相对基址变址寻址。

存储器寻址方式

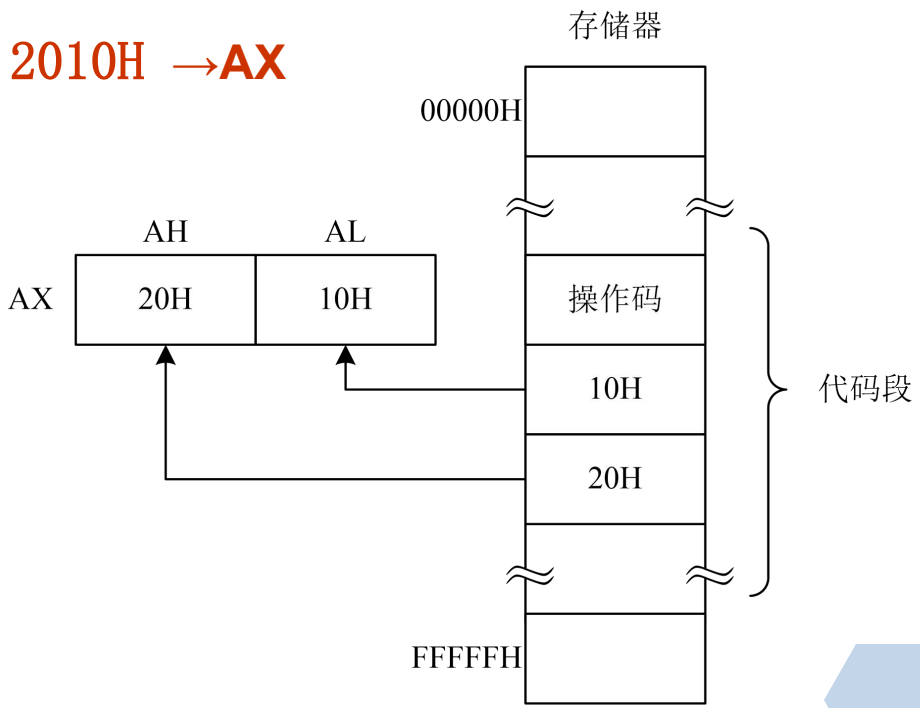
4.2.1 立即寻址

- 操作数直接出现在指令中。
- 操作数存放在代码段中，并紧跟在操作码后面。

例：① MOV AL，1AH；1AH →AL

② MOV AX，2010H；2010H →AX

错误 MOV AL，300；



约束条件

① 在所有的指令中，立即数只能作源操作数，不能作目标操作数。
② 立即数应与目标操作数的长度一致。
③ 立即数默认采用十进制形式，以十六进制形式出现的立即数应以字母H为后缀，以八进制形式出现的立即数应以字母Q为后缀。
④ 以十六进制形式出现的立即数，若以字母开头，则必须以数字0为前缀。
⑤ 立即数还可以用表示+、-、×、/表示的算术表达式，也可以用圆括号改变运算顺序。
⑥ 立即数只能是整数，不能是小数、变量或其它类型的数据。

4.2.2 寄存器寻址

- 操作数在指令指定的寄存器中。
- 操作数在CPU内部的寄存器中，指令执行时，不需要通过访问存储器来取得，因而指令的执行速度快。
- 由于寄存器号短，因此，采用寄存器寻址方式的指令的机器码长度短。

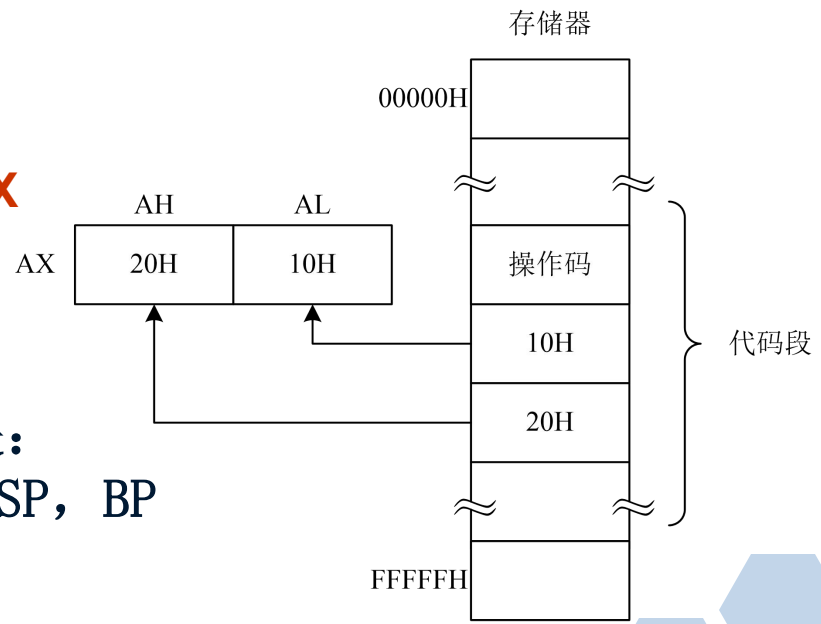
例如：

MOV AX，2010H；2010 → AX

ADD AX，10 ；（AX）+10 → AX

INC CX ；（CX）+1 → CX

MOV AL，CL ；（CL）→ AL



- 对于16位操作数，寄存器可以是：
  - AX，BX，CX，DX，SI，DI，SP，BP
  - CS，DS，SS，ES
- 对8位操作数，寄存器可以是：
  AH，AL，BH，BL，CH，CL，DH，DL

约束条件

① 在一条指令中，寄存器寻址方式既可用于源操作数，也可用于目标操作数，还可以两者都用寄存器寻址方式。

② 源操作数与目标操作数的长度应一致。例如，不能将寄存器AX的内容传送到寄存器BH中，也不能将寄存器BH的内容传送到寄存器AX中。

③ 两个操作数不能同时为段寄存器。

④ 目标操作数不能是代码段寄存器。

4.2.3 直接寻址

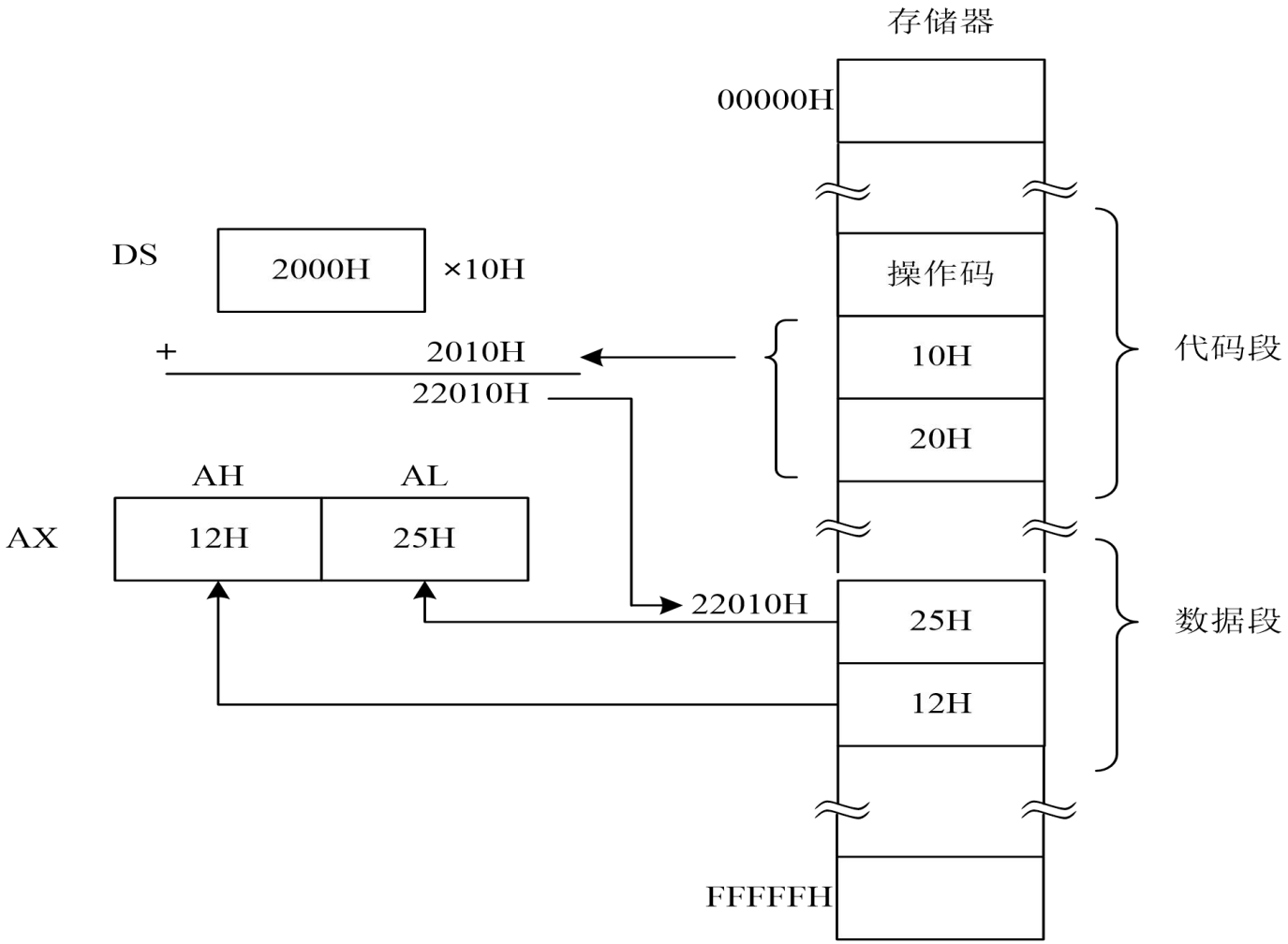
- 指令中直接给出操作数在内存的有效地址EA。为了与立即数区别，该有效地址必须用[]括起。
- 操作数在存储器中

例如：

MOV AX，[2010H]；（DS：2010H）→ AL
（DS：2011H）→ AH

MOV AX，ES：[10H]；（ES：0010H）→ AL
（ES：0011H）→ AH

MOV AL，LABLE；（DS：LABLE）→ AL，其中LABLE是源程序中用户定义的地址变量，它将在编译过程中变换成具体的偏移地址值。


存储器
00000H
DS
2000H
×10H
+
2010H
22010H
操作码
10H
20H
代码段
AH
AL
AX
12H
25H
22010H
25H
12H
数据段
FFFFFH

约束条件及说明

① 指令中没有用前缀说明操作数存放在哪个段，则操作数默认存放在数据段。

② 8086系统允许操作数存放在代码段、堆栈段或附加段。此时，就需要在指令中指明段超越。

③ 在汇编语言指令中，可以用符号地址代替数值地址。

4.2.4 寄存器间接寻址

- 操作数的有效地址EA存放在寄存器中，指令中指定的寄存器名要用[]括起来，可用寄存器：BX、BP、SI、DI。
- 操作数在内存，使用SI、DI、BX寄存器时，操作数默认存放在数据段中；使用BP寄存器时，操作数默认存放在堆栈段中，允许段超越。

① MOV AX，[BX]；　（DS：EA）→ AL
　　（DS：EA+1）→ AH，其中：EA =（BX）
② MOV AL，ES：[BX]；（ES：EA）→ AL，其中： EA =（BX）
③ MOV AX，[BP]；　（SS：EA）→ AL
　　（SS：EA+1）→ AH，其中：EA =（BP）

例如：

MOV AX，[SI]

解：若

(DS)=2000H

(SI)=2010H，

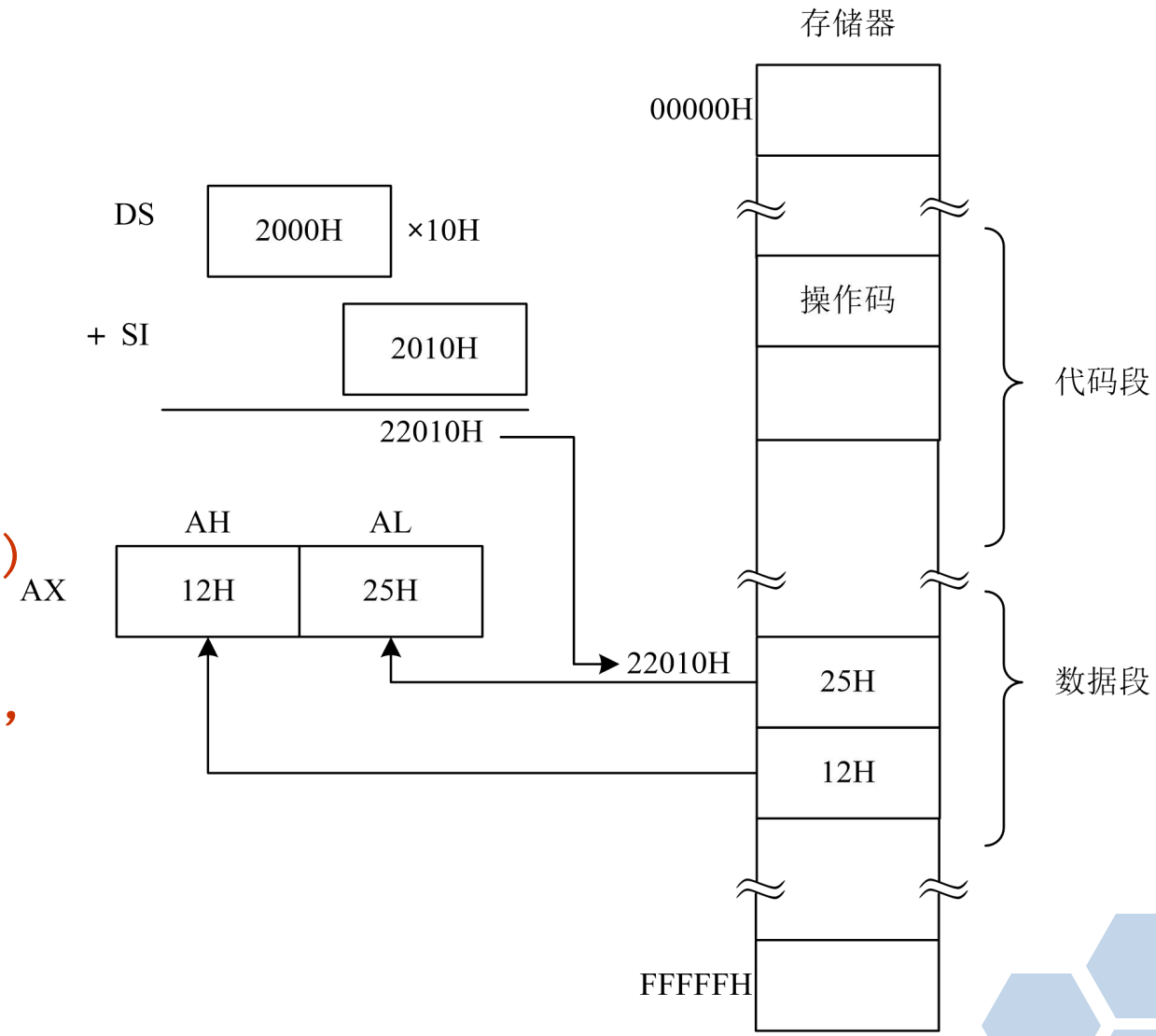
操作数物理地址

=(DS)×10H+(SI)

=22010H

那么指令执行后，

(AX)=1225H。



4.2.5 寄存器相对寻址

- 操作数的有效地址EA是指令中指定的基址或变址寄存器的值与位移量之和，可用寄存器：BX、BP、SI、DI。
- 操作数在内存，使用SI、DI、BX寄存器时，操作数默认存放在数据段中；使用BP寄存器时，操作数默认存放在堆栈段中，允许段超越。
- 位移量是符号数，8位偏移量的取值范围为：00～0FFH(即+127D～-128D)；
- 16位位移量的取值范围为：0000～0FFFFH(即+32767D～-32768D)。
- 8086汇编允许用下面三种形式表示相对寻址，它们是等效的。

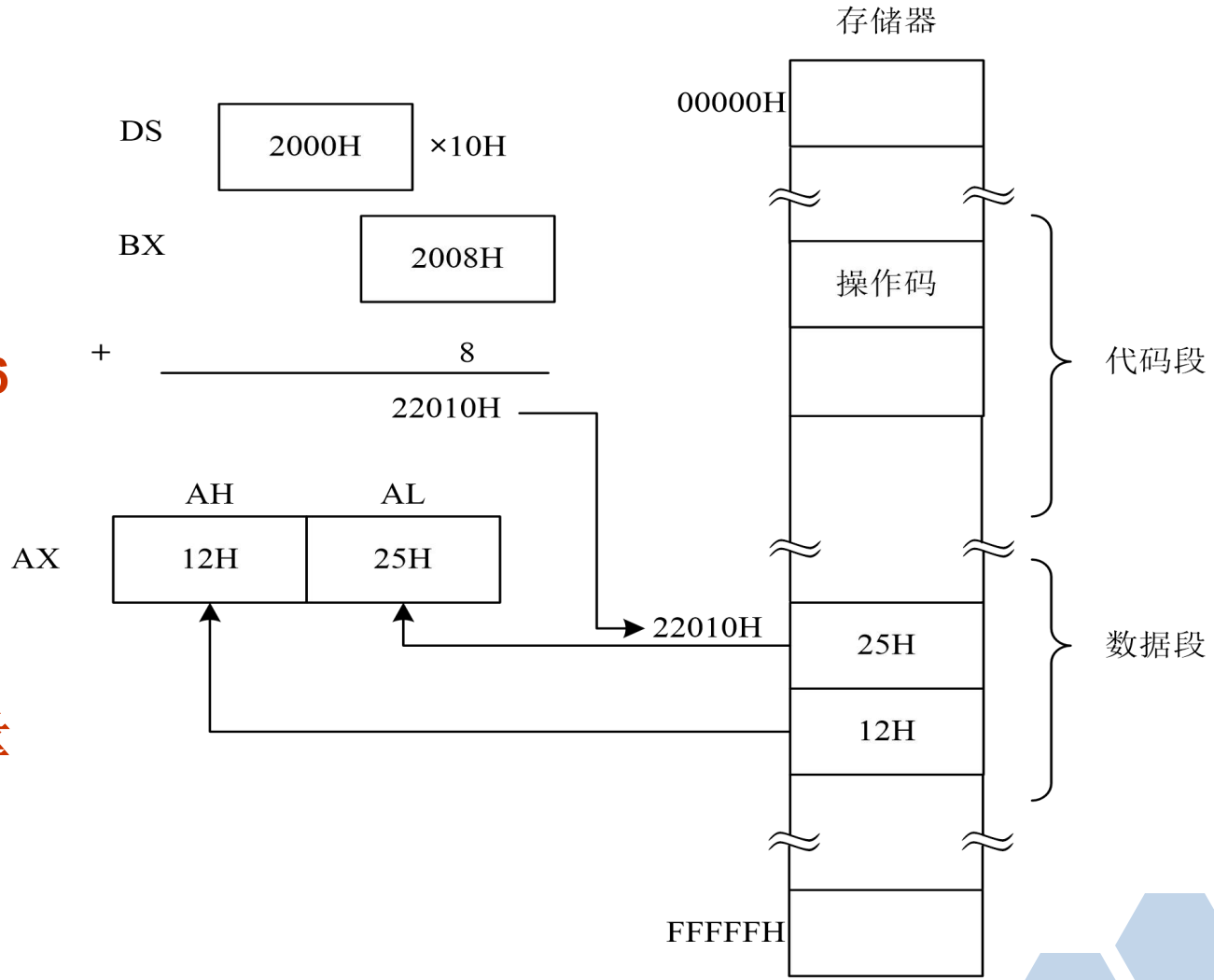
```
MOV AX，[BX]+8
MOV AX，8[BX]
MOV AX，[BX+8]
```

操作数的物理地址

=(DS)×10H+(SI)/(DI)/(BX)+8位或16位位移量

或

=(SS)×10H+(BP)+8位或16位位移量



4.2.6 基址变址寻址

- 操作数的有效地址EA是指令中指定的基址寄存器的值与变址寄存器的值之和
- 操作数在内存，指令中使用基址寄存器BX时，操作数默认存放在数据段中；使用基址寄存器BP时，操作数默认存放在堆栈段中，允许段超越。
- 操作数的物理地址
  =（DS）×10H+（SI）/（DI）+（BX）
  或
  =（SS）×10H+（SI）/（DI）+（BP）

例如：

MOV AX，[BX] [SI]

解：

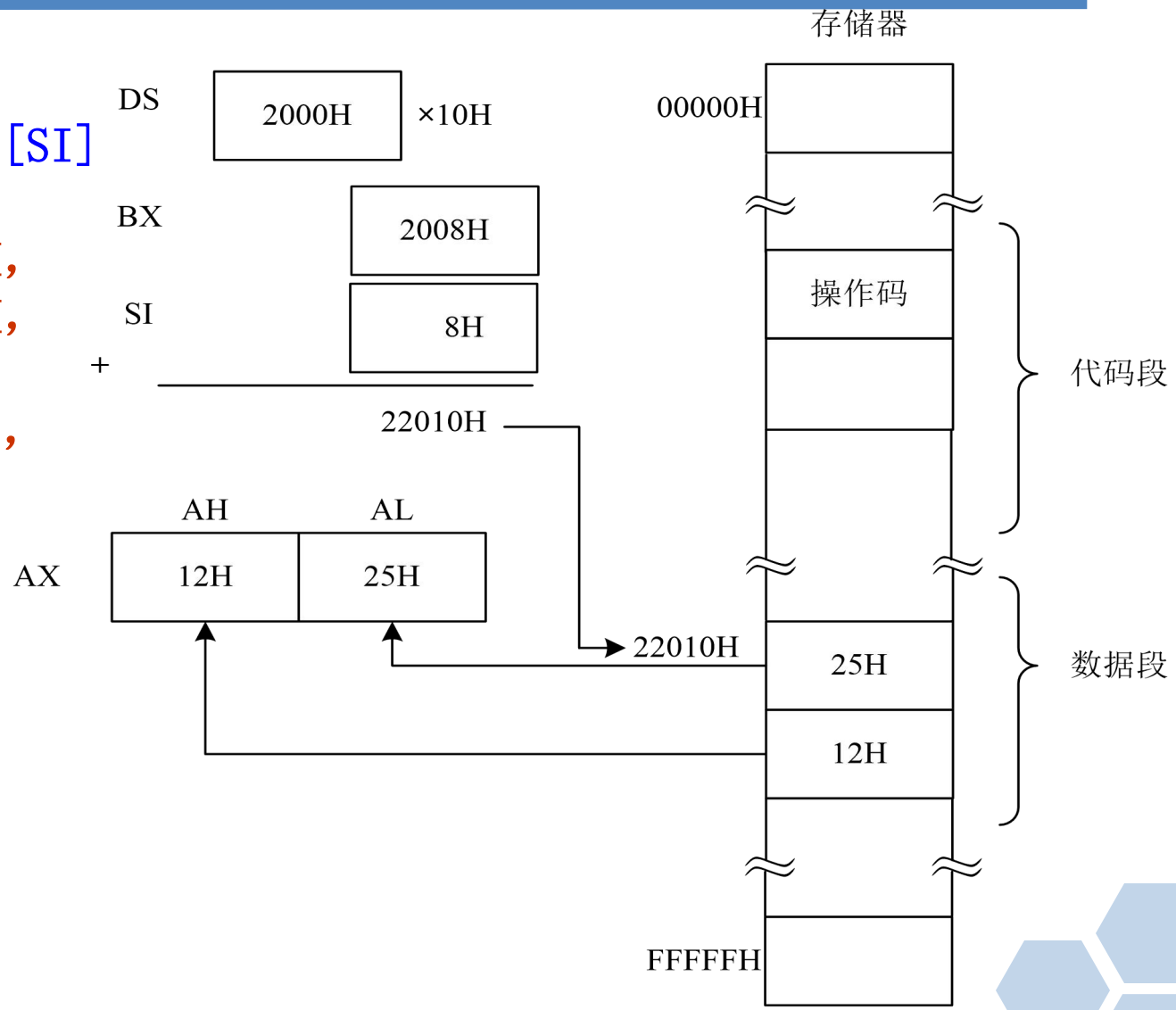
若（DS）=2000H，

（BX）=2008H，

（SI）=8H，

那么指令执行后，

（AX）=1225H。



4.2.7 相对基址变址寻址

- 操作数的有效地址EA是指令中指定的基址寄存器的值与变址寄存器的值以及8位或16位移量之和。
- 操作数在内存，指令中使用基址寄存器BX时，操作数默认存放在数据段中；使用基址寄存器BP时，操作数默认存放在堆栈段中，允许段超越。
- 操作数的物理地址
  =（DS）×10H+（SI）/（DI）+（BX）+8位或16位位移量
  或
  =（SS）×10H+（SI）/（DI）+（BP）+8位或16位位移量

例如：

MOV AX，3[BX][SI]

解：若

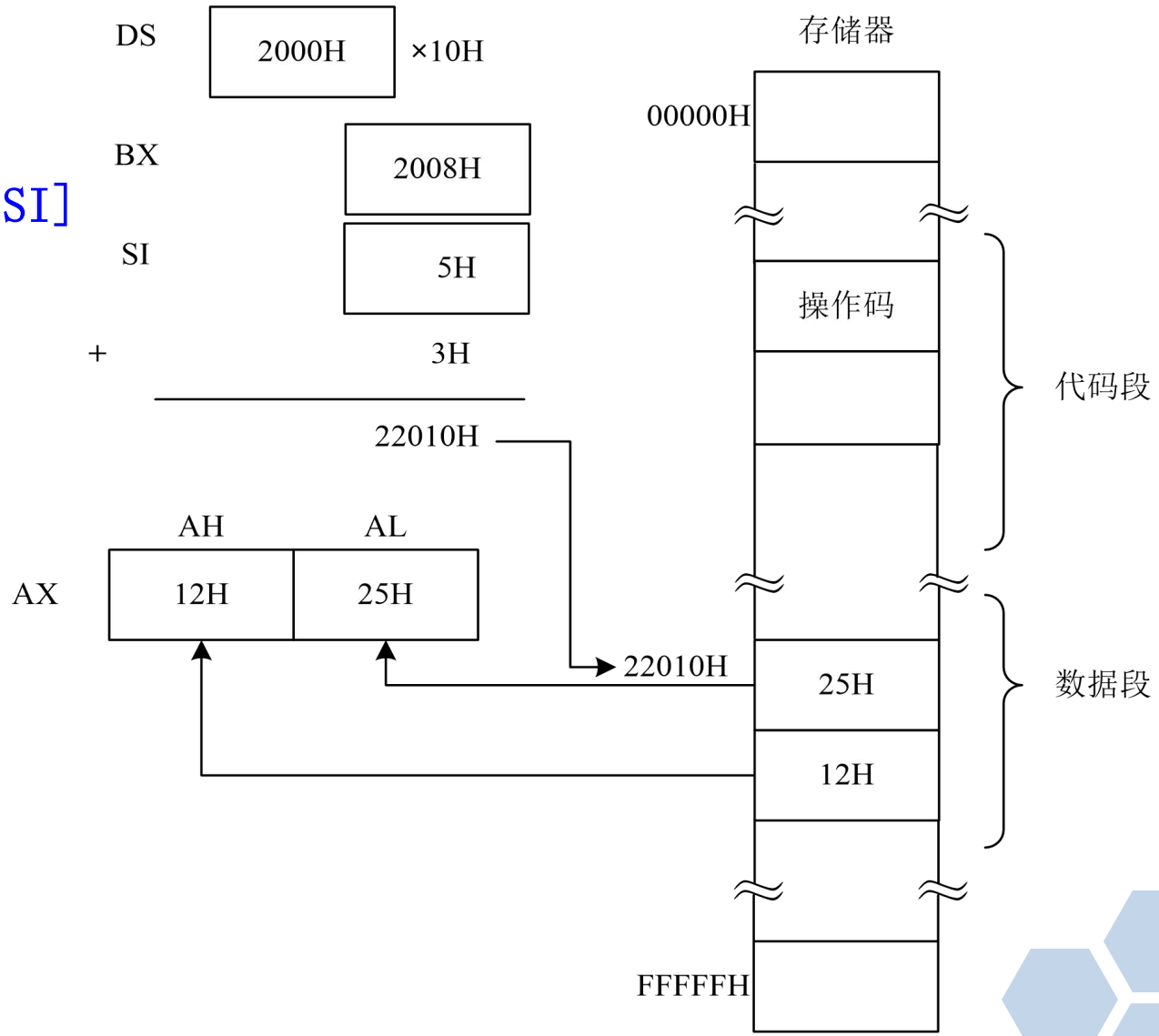
（DS）=2000H，

（BX）=2008H，

（SI）=5H，

那么指令执行后，

（AX）=1225H。



课练： 在8086系统中，设DS=1000H，ES=2000H，SS=1200H， BX=0300H， SI=0200H， BP=0100H， VAR的偏移量为0060H，请指出下列指令的目的操作数的寻址方式，若目的操作数为存储器操作数，请计算它们的物理地址是多少？

- MOV BX,12
- MOV [BX],12
- MOV ES:[SI],AX
- MOV VAR,8
- MOV [BX][SI],AX

⑥ MOV 6[BP][SI],AL

⑦ MOV [1000H],DX

⑧ MOV 6[BX],CX

⑨ MOV VAR+5,AX

解：

（1）寄存器寻址方式
（2）寄存器间接寻址方式，PA=10300H
（3）寄存器间接寻址方式，PA=20200H
（4）直接寻址方式，PA=10060H
（5）基址变址寻址方式，PA=10500H
（6）相对基址变址寻址方式，PA=12306H
（7）直接寻址方式，PA=11000H
（8）寄存器相对寻址方式，PA=10306H
（9）直接寻址方式，PA=10065H

4.3 8086指令系统

8086指令系统包括六大类指令：

① 数据传送指令

② 算术运算指令

③ 逻辑运算和移位指令

④ 串操作指令

⑤ 控制转移指令

⑥ 处理器控制指令

数据传送指令的共同特点

① 除了POPF和SAHF指令外，其他的数据传送指令的执行结果都不影响标志位。

② 指令中如果列出两个操作数，则指令的执行过程是：目标操作数←源操作数。

③ 指令中如果仅列出一个操作数，则另一个操作数为隐含操作数。

4.3.1数据传送指令

- 数据传送指令是将数据或地址传送到寄存器、存储单元或I/O端口中。分为5类：

  ① 通用数据传送指令；

  ② 累加器专用传送指令；

  ③ 地址传送指令；

  ④ 标志传送指令；

  ⑤ 数据类型转换指令。

1. 通用数据传送指令

MOV 指令族（Move）—数据传送指令

- 操作：将源操作数送目的操作数
- 标志：所有标志不受影响。

说明：注意MOV指令在任何情况下均不影响标志。

例：
```
MOV AX，BX
MOV CL，DH
MOV DX，DS
MOV DX，[SI+100]
MOV BX，84
MOV DS：[BP]，3989
MOV WORD PTR [BX+SI+100]，DI
```

堆栈

① 堆栈是一块按照“后进先出”原则工作的内存区域。
② 把数据从栈顶存入堆栈中的操作称为入栈（或压入）；
③ 把数据通过栈顶从堆栈中取出的操作称为出栈（或弹出）。
④ 堆栈常被用于数据的暂存、交换、子程序的参数传递等场合。
⑤ 在调用子程序或中断服务程序时，堆栈是默认的被用于保存返回地址的内存区域。
⑥ 为了实现子程序或中断嵌套，也必须使用堆栈技术。

堆栈

⑦ 在8086系统中，堆栈所在的段就是堆栈段，它可以占用的最大空间是64KB。

⑧ 堆栈段的段地址由SS寄存器指示，堆栈指针寄存器SP始终指示栈顶的偏移地址并随着入栈和出栈操作而自动变化。

⑨ 当进行压入操作后堆栈指针达到定义值，表明堆栈满；当执行弹出操作后堆栈指针回到初值，表明堆栈空。

⑩ 当栈满时，再压入数据，称为“堆栈溢出”。

PUSH 指令（Push word onto stack）

- 操作：将指定的操作数内容推入堆栈，且栈指针SP减2。

  （SP）←（SP）-2

  （SP）-1←操作数高字节

  （SP）-2←操作数低字节

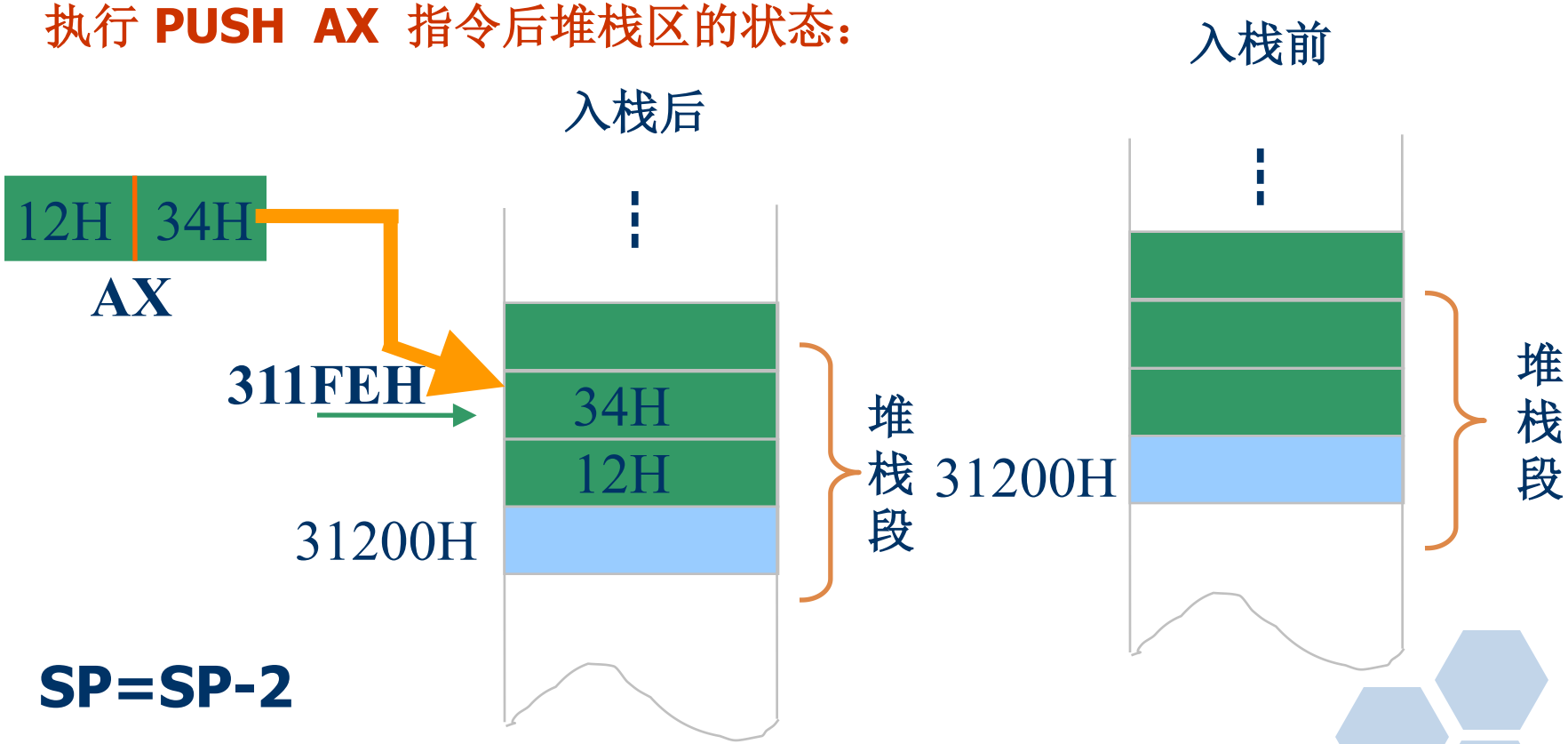
- 约束：PUSH指令总是把2个字节一起推入堆栈，
  PUSH AL指令是非法的；

例：PUSH AX
PUSH ES
PUSH WORD PTR BX

PUSH AX

设AX=1234H，SP=1200H， SS=3000

执行 PUSH AX 指令后堆栈区的状态：



SP=SP-2

POP 指令（Pop word off stack into destination）

- 操作：将堆栈顶部2个字节弹出，栈指针SP加2。

  （SP） ← 操作数低字节

  （SP）+1 ← 操作数高字节

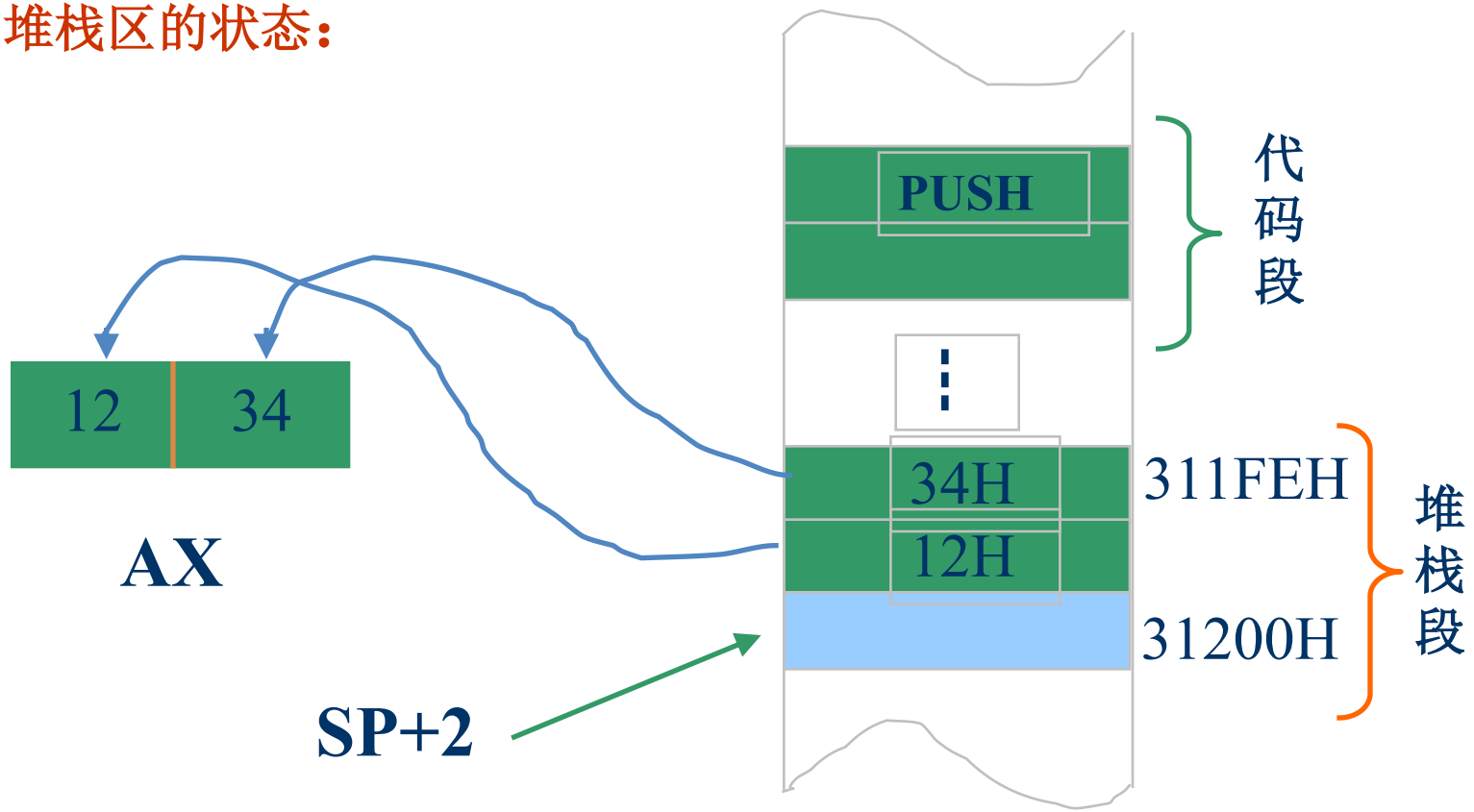
  (SP) ← （SP）+2

- 约束：使用POP指令时，总是弹出2个字节，而不能只弹出1个字节。此外，POP CS是非法指令。

例：POP CX
POP DS
POP [BX+100]

POP AX

堆栈区的状态：



XCHG 指令（Exchange）—交换

- 操作：目的操作数的内容和源操作数的内容交换
- 约束：XCHG不能对段寄存器进行内容交换。

例：XCHG AX，BX
XCHG WORD PTR [DI]，CX
XCHG BL，[2000]

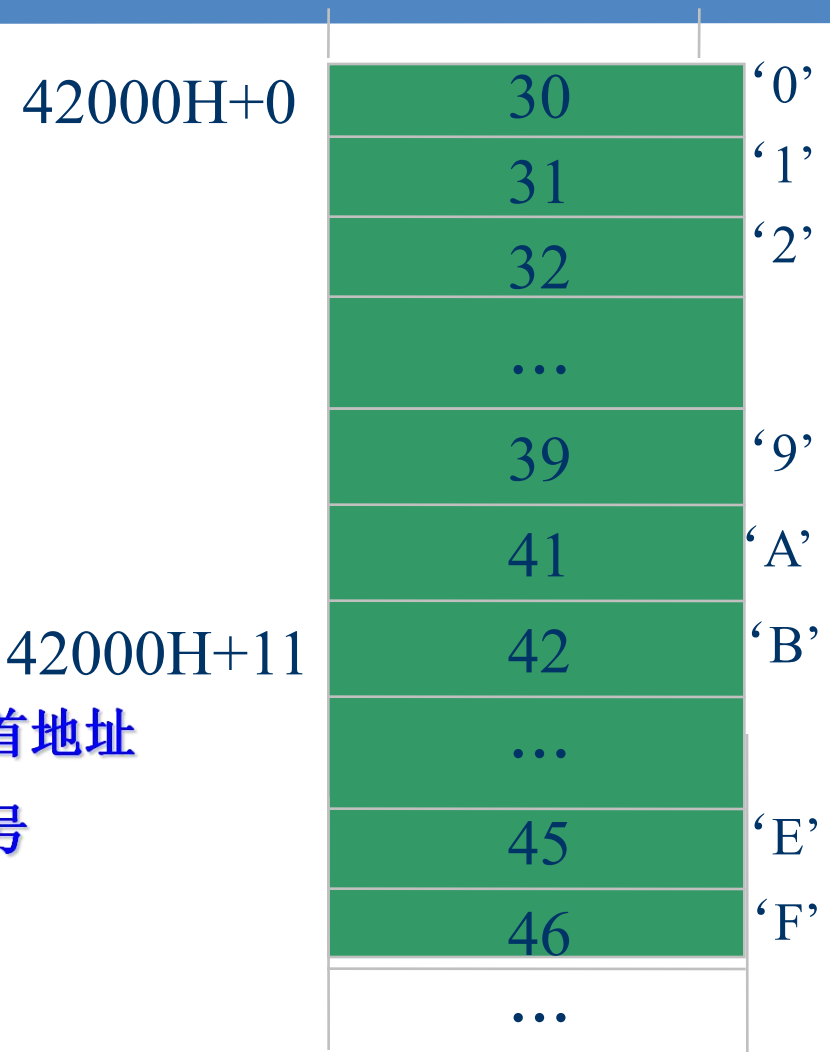
2.累加器专用传送指令

XLAT 指令（Translate）——换码

- 操作：把数据段中偏移地址为BX+AL的内存单元的内容送到AL中，即：(AL) ← (BX + AL)。
- 说明：将内存表格中某一单元的值传送至寄存器AL，实现一种编码到另一种编码的转换。例如，把字符的扫描码转换成ASCII码。
- 约束：
  - 寄存器**BX**保存内存表格的首地址；寄存器**AL**保存表格中某单元在此表格中的偏移量。
  - 该指令能访问的内存表格中的数据只能是字节类型的。
  - 该指令能访问的内存表格的最大容量是**256**字节。

查表指令例

数据段中存放有一张ASCII码转换表，设首地址为42000H，现欲查出表中第11个代码的ASCII码。



可用如下指令实现：

```
MOV  DS，4000H
MOV  BX，2000H ；（BX）←表首地址
MOV  AL，0BH   ；（AL）←序号
XALT           ；查表转换
```

执行后得到：（AL）=42H

3. 地址传送指令

LEA 指令（Load effective address）—地址传送

操作：把变量、标号或表达式所指的内存单元的偏移地址(有效) 送指定寄存器。

约束：要求源操作数必须为存储器地址，目的操作数必须为16位的通用寄存器。

例：

```
LEA  BX，TABLE
LEA  SI，TABLE +10
LEA  DI，[BX+SI+100]
```

LDS 指令（Load data segment register）—地址传送

操作：将32位地址(逻辑地址)的高16位送DS，低16位送指定寄存器。
约束：指令中源操作数是存储器某4个连续单元的偏移地址，目标操作数是间址寄存器。

例：

LDS BX，[SI+100]

LES 指令（Load data segment register）—地址传送

操作：将32位地址(逻辑地址)的高16位送ES，低16位送指定寄存器。

约束：指令中源操作数是存储器某4个连续单元的偏移地址，目标操作数是间址寄存器。

例：

LDS BX，[SI+100]

LDS指令例： LDS DI，[1200H]

MOV AX，[DI]

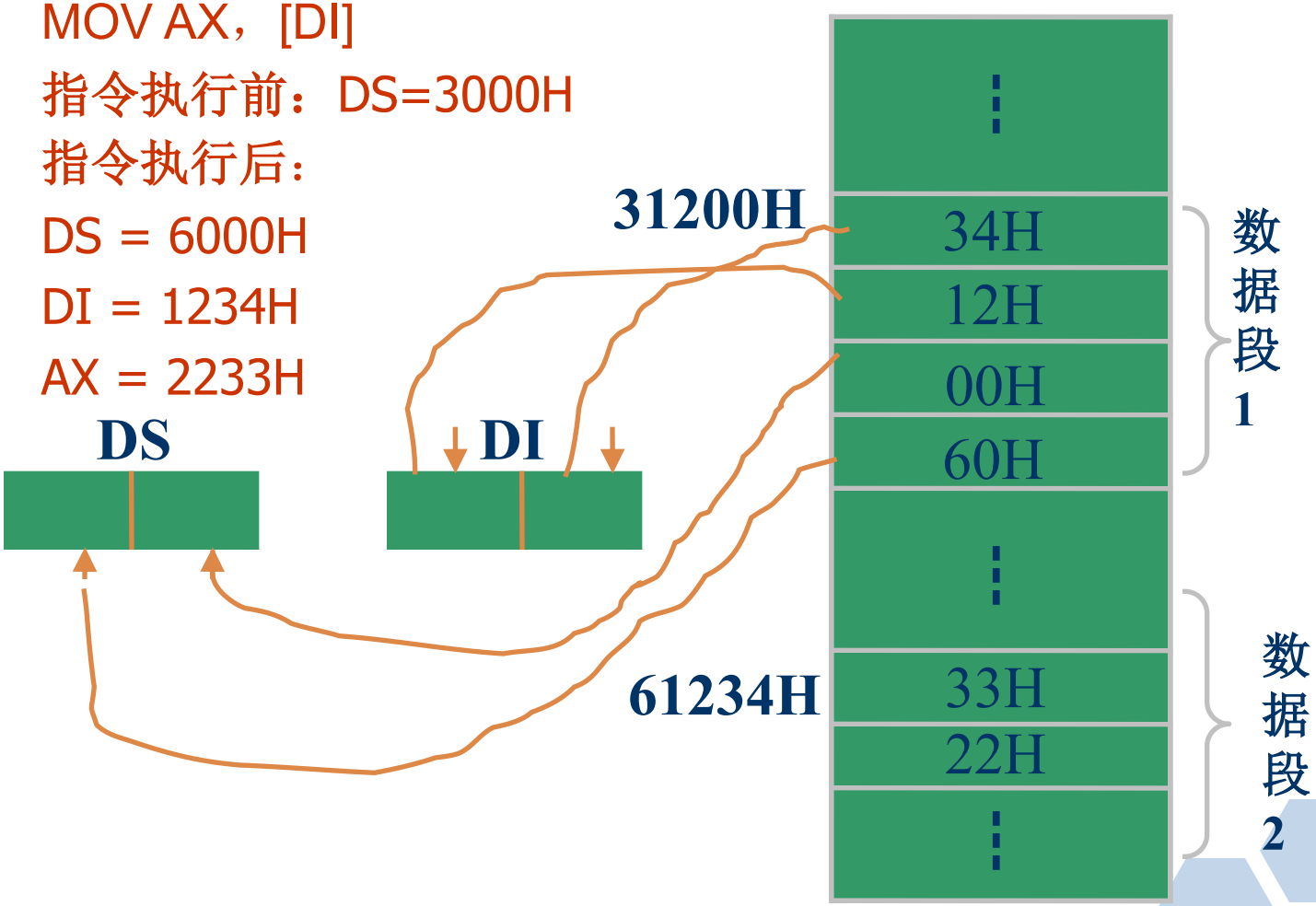
指令执行前：DS=3000H

指令执行后：

DS = 6000H

DI = 1234H

AX = 2233H



4.标志传送指令

LAHF 指令（Load AH from flags）——取标志指令

操作：将标志传送到AH寄存器中

格式如下：

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
SF	ZF		AF		PF		CF

说明：LAHF指令传送标志时，第1、3、5位的值不确定。

SAHF 指令（Store AH onto flags）—置标志指令

操作：将AH的内容写入标志寄存器7~0位。

格式如下：

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
SF	ZF		AF		PF		CF

标志：影响AF、CF、PF、SF、ZF

说明：SAHF指令只改变标志寄存器的低8位，而不影响高8位。

标志寄存器是特殊寄存器，不能像一般数据寄存器那样随意操作，以免其中的值发生变化。

PUSHF指令——标志入栈指令

功能：标志寄存器入栈。

① SP ← SP–2；

② (SP + 1，SP) ← FLAG。

- 该指令中的源操作数隐含为标志寄存器，目标操作数隐含为堆栈区。

D_{15}	D_{14}	D_{13}	D_{12}	D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
				OF	DF	IF	TF	SF	ZF		AF		PF		CF

POPF指令——标志出栈指令

功能：数据出栈到标志寄存器。

① FLAG ← (SP+1，SP)；

② SP ← SP + 2。

- 该指令中的源操作数隐含为堆栈区，目标操作数隐含为标志寄存器。
- SAHF和POPF指令直接改变标志寄存器的值。利用这一特性，可以非常方便地改变有关的标志位。

例：编写汇编程序段，把标志寄存器的CF位求反，其他位不变。

```
LAHF                ；取标志寄存器的低8位
XOR   AH,01H        ；最低位求反,其他位不变
SAHF                ；送入标志寄存器的低8位
```

例：编写汇编程序段，把标志寄存器TF位清零，其他标志位不变。

```
PUSHF               ；标志寄存器入栈
POP   AX            ；取标志寄存器内容
AND   AX,0FEFFH     ；TF清零,其他位不变
PUSH  AX            ；新值入栈
POPF                ；送入标志寄存器
```

5. 数据类型转换指令

CBW指令——字节转换为字指令

功能：把寄存器AL中数据的符号位扩到AH寄存器中，使字节转换为字。指令的执行如下：

当AL<80H时，AH←00H；

当AL≥80H时，AH←FFH。

- 说明：一个用补码表示的数经CBW指令进行符号位扩展后，数值大小不变。

5. 数据类型转换指令

CWD指令——字转换为双字指令

功能：把寄存器AX中数据的符号位扩到DX寄存器中，使字转换为双字。指令的执行如下操作：

当AL<8000H时，DX←0000H；

当AL≥8000H时，DX←FFFFH。

- 说明：一个用补码表示的数经CWD指令进行符号位扩展后，数值大小不变。

例如：

（1） MOV AL，75H
CBW ；执行结果为：AX=0075H

（2） MOV AX，0A085H
CWD ；执行结果为：DX=0FFFFH,AX=0A085H

4.3.2算术运算指令

算术运算类指令：

- 加法运算指令
- 减法运算指令
- 乘法指令
- 除法指令

约束：

- 算术运算指令的执行大多对状态标志位会产生影响。
- 算术运算指令中的操作数有一个或两个。
- 双操作数指令中，除了源操作数是立即数这种情况外，其余情况下，必须有一个操作数在寄存器中。
- 单操作数指令中的操作数不允许是立即数。

1. 加法指令

- ADD 加法指令
- ADC 带进位位的加法指令
- INC 加1指令

ADD 指令（Addition）—加法（不带进位）

- 操作：将两个操作数相加，再把和送到目的操作数中。
- 标志：AF、CF、OF、PF、SF、ZF受影响
- 说明：如果遇到要把寄存器字或存储器字和一个立即数字节相加，那么，加之前，先要把立即数字节扩展为字（16位）。

例：

```
ADD  AX，BX
ADD  DX，[BX+DI+200]
```

ADC 指令（Add with carry ）—带进位的加法

- 操作：在进行加法时，将CF加如和中。
- 标志：**AF、CF、OF、PF、SF、ZF**受影响
- 说明：ADC指令多用于多字节数相加，使用前要先将CF清零。

  如果寄存器字或存储器字要和立即数字节相加，那么，在加之前，要先进行符号扩展，使立即数字节成为立即数字。

INC 指令（Increment destination by 1）—增量

- 操作：将指定的操作数加1
- 标志：AF、OF、PF、SF、ZF受影响，CF? 无意义。
- 说明：INC指令将目的操作数加1，再送回结果，要特别注意的是INC指令不会影响CF，就是说，最高位不会产生进位。

2. 减法指令

- SUB　减法指令
- SBB　考虑借位的减法指令
- DEC　减1指令
- CMP　比较指令
- NEG　求补指令

SUB指令（Subtract）—减法

- 操作：本指令执行目的操作数减源操作数动作，结果在目的操作数中。
- 标志：影响AF、OF、CF、PF、SF、ZF。
- 说明：当从寄存器/存储器字中减去1个立即数字节时，必须先把此字节扩展成16位立即数。

SBB 指令（Add with carry ）—带借位的减法

- 操作：在进行减法时，同时减去CF。
- 标志：AF、CF、OF、PF、SF、ZF受影响
- 说明：SBB指令多用于多字节数相减，使用前要先将CF清零。

DEC 指令（Decrement destination by one）—减1操作

- 操作：把指定的操作数减1。
- 标志：AF、OF、PF、SF、ZF受影响，CF无意义。
- 说明：DEC指令将指定操作数减1，并将结果送回操作数。

CMP 指令（Compare two operands）—比较两个操作数

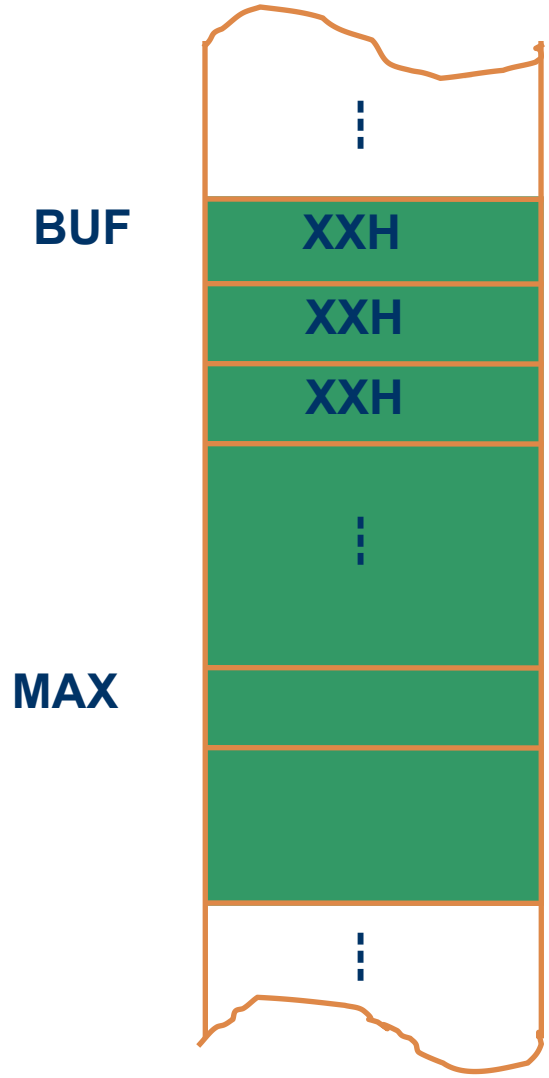
- 操作：从目的操作数（左边）中减去源操作数(右边)，但结果不送回，而只按减的结果影响标志。
- 标志：AF、CF、OF、PF、SF、ZF受影响。
- 说明：使用CMP指令时，源操作数（右边的）和目的操作数（左边的）的类型（字或字节）要相同。

课练：CMP指令例

```
        LEA BX，MAX
        LEA SI，BUF
        MOV CL，20
        MOV AL，[SI]
NEXT：  INC  SI
        CMP AL，[SI]
        JNC  GOON       ；CF=0转
        XCHG  [SI]，AL
GOON：  DEC  CL
        JNZ  NEXT
        MOV [BX]，AL
        HLT
```

程序功能？

从BUF开始的数据表中（20个数据中）找出最大的数，并将其存放在MAX单元中



NEG 指令（Negate,or form 2’s complement）—求补

- 操作：对操作数求补后存回指定的地址。
- 标志：AF、CF、OF、PF、SF、ZF受影响。
- 说明：NEG指令用来求一个数的补数。

3 .乘法指令

- 无符号的乘法指令MUL
- 带符号的乘法指令IMUL

约束：

- 乘法指令采用隐含寻址，隐含的是存放被乘数的累加器AL或AX及存放结果的AX，DX。
- 指定的操作数不能是立即数。

MUL 指令（Multiply accumulator by register-or-memory；unsigned）—无符号数的乘法

- 操作：用指定的操作数和累加器中数（若为字节，则为AL；若为字，则为AX）相乘，对于两个8位无符号数的乘法，16位的乘积在AH和AL中，对于两个16位无符号数的乘法，32位的乘积在DX和AX中。
- 标志：CF、OF 受影响, AF、PF、SF、ZF 不受影响。
- 说明：DX和AH中分别为乘积高位，

  如乘积高位不为0，CF和OF为1，表示结果为有效数；

  若运算结果OF=CF=0，表示其为无效数据。

IMUL 指令（Integer multiply accumulator by register-or-memory；signed ）—带符号的整数乘法

- 操作：将累加器AL 或AX 的内容乘以指定的操作数，乘积在AX 中或者在DX：AX 中。
- 标志：CF、OF 受影响，AF、PF、SF、ZF 无意义。
- 说明：

如果乘积的高一半为低一半的符号扩展，则CF 和OF 置0，表示其为无效数据。

当CF 和OF 为1 时，说明AH 或DX 中的内容是乘积的高位。

4. 除法指令

无符号除法指令 **DIV OPRD**
有符号除法指令 **IDIV OPRD**

若是字节运算，则执行**AX/OPRD**
- **AL=商**
- **AH=余数**

若是字运算，则执行 DX:AX/OPRD
- **AX=商**
- **DX=余数**

约束：指令要求被除数是除数的双倍字长

5.BCD码调整指令

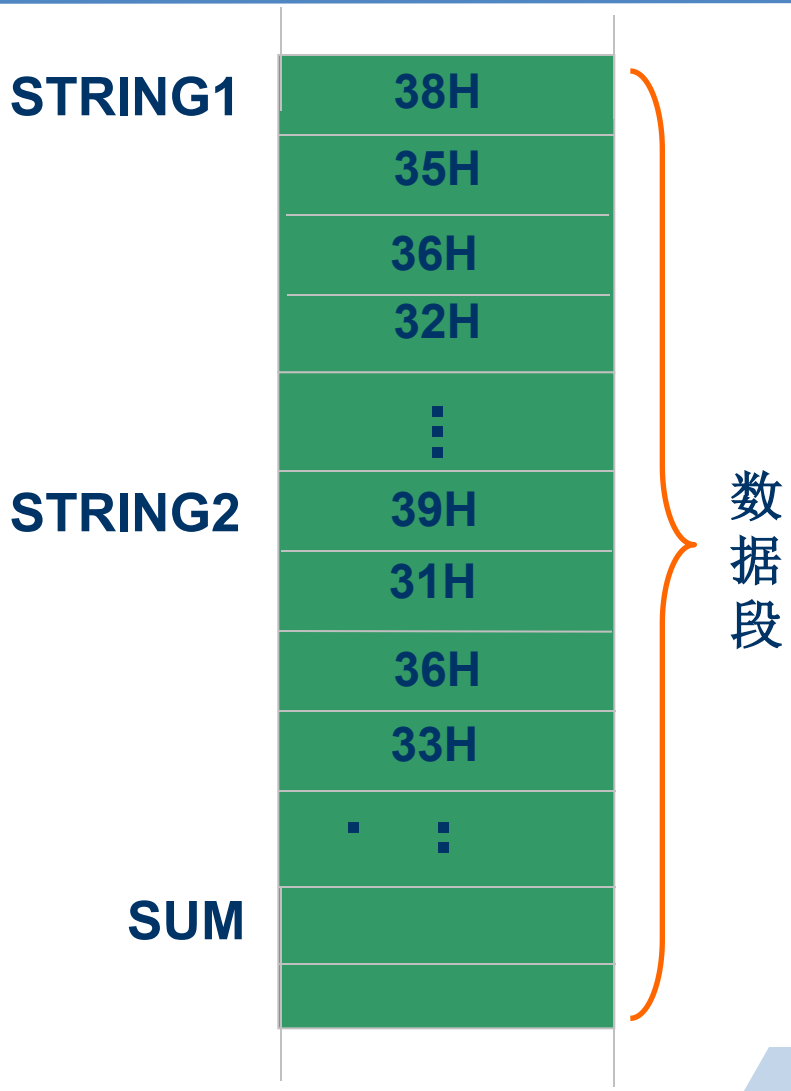
DAA
DAS
} 压缩BCD码的加、减法调整

AAA
AAS
AAM
AAD
} 非压缩BCD码的加、减、乘、除法调整

- 共6条，均为隐含寻址方式，隐含的操作数是AL或AL、AH；
- 不能单独使用，要紧跟在相应的算术运算指令之后；
- 执行结果为压缩BCD码或扩展BCD码表示的十进制数。

课练：加法指令应用

设被加数和加数的每一位都以ASCII码形式存放在内存中，低位在前，高位在后。计算2658+3619=？结果存放在SUM单元中。



加法指令应用例

```
        LEA  SI，STRING1
        LEA  DI，STRING2
        LEA  BX，SUM
        MOV CX，4
        CLC
NEXT：  MOV  AL，[SI]
        ADC  AL，[DI]
        AAA
        MOV  [BX]，AL
        INC  SI
        INC  DI
        INC  BX
        DEC  CX
        JNZ  NEXT
```

问：从SUM开始连续的4个单元的内容？修改？

4.3.3逻辑运算指令

- 逻辑运算
  - 与
  - 或
  - 非
  - 异或
- 移位操作
  - 非循环移位
    - 逻辑移位
    - 算术移位
  - 循环移位
    - 不带进位位的移位
    - 带进位位的移位

1. 逻辑运算

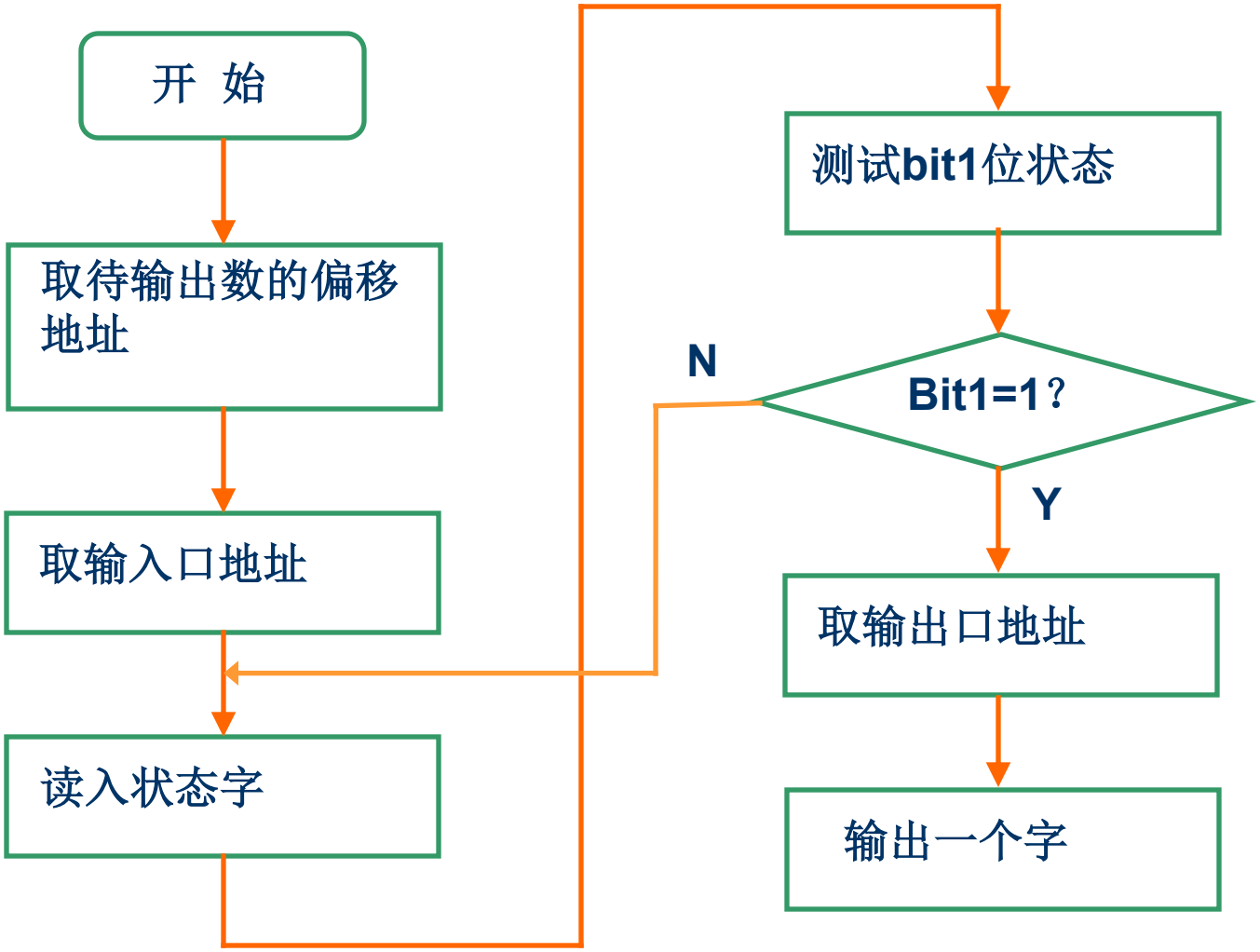
AND　逻辑与
OR　或
NOT　非
XOR　异或
TEST　测试指令

- NOT指令不影响标志；
- 其他指令：CF,OF为0；AF无意义；
  SF,ZF,PF由运算结果确定。
- 说明：“非”运算指令要求操作数不能是立即数

指令的应用例

AND 例　屏蔽4、7两位
OR　例　要求第7位置1，其他位不变
XOR 例　将AL中的最高位变反

指令的应用例　输出设备的状态查询和数据输出



指令的应用例

```
        LEA     SI，DATA
        MOV     DX，38F0H
WAT：   IN      AL，DX
        TEST    AL，02H
        JZ      WAT          ；ZF=1转移
        MOV     DX，38FEH
        MOV     AX，[SI]
        OUT     DX，AX
```

2. 移位

SHL　逻辑左移
SAL　算数左移
SHR　逻辑右移
SAR　算数左移
ROL　循环左移
ROR　循环右移
RCL　带进位循环左移
RCR　带进位循环右移

标志：查表。

- 说明：移动一位时由指令直接给出；移动两位及以上，则移位次数由CL指定。
- 应用：
  ①用于对某些位状态的测试
  ②高位部分和低位部分的交换
  ③与非循环移位指令一起组成32位或更长字长数的移位

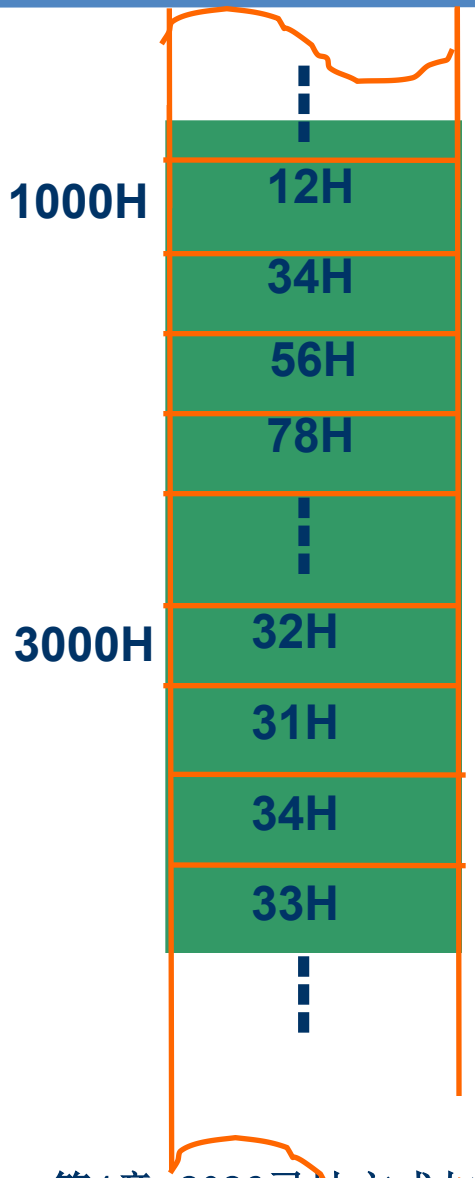
课练：分析程序

```
        MOV SI,1000H
        MOV DI,3000H
        MOV CX,4
BBB:    MOV AL,[SI]
        MOV BL,AL
        AND AL,0FH
        OR AL,30H
        MOV [DI],AL
        INC DI
        MOV AL,BL
        PUSH CX
        MOV CL,4
        SHR  AL,CL
        OR   AL,30H
        MOV  [DI],AL
        INC  DI
        INC SI
        POP CX
        DEC CX
        JNZ BBB
        HLT
```

程序功能？

解：程序功能描述

将1000H开始存放的四个压缩BCD码转换为ASCII码存放在3000H开始的单元中去。



4.3.4 串操作指令

MOVS串传送

CMPS串比较

SCAS串扫描

LODS串加载

STOS串存储

- 针对数据块或字符串的操作
- 待操作的数据串称为源串，目标地址称为目标串

串操作指令的特点：

- 源串一般存放在数据段，偏移地址由SI指定,允许段重设。
- 目标串必须在附加段，偏移地址由DI指定。
- 指令自动修改地址指针，修改方向由DF决定。

  DF=0 增地址方向；DF=1 减地址方向。
- 数据块长度值由CX指定。
- 可增加自动重复前缀以实现自动修改CX内容。

MOVSB/MOVSW 指令（Move byte string or move word string）

- 功能：将源字节串（或字串）传送给目的字节串（或字串）。
- 说明：源字节串（字串）的地址由DS：SI 提供，

  目的字节串（字串）的地址由ES：DI 提供。

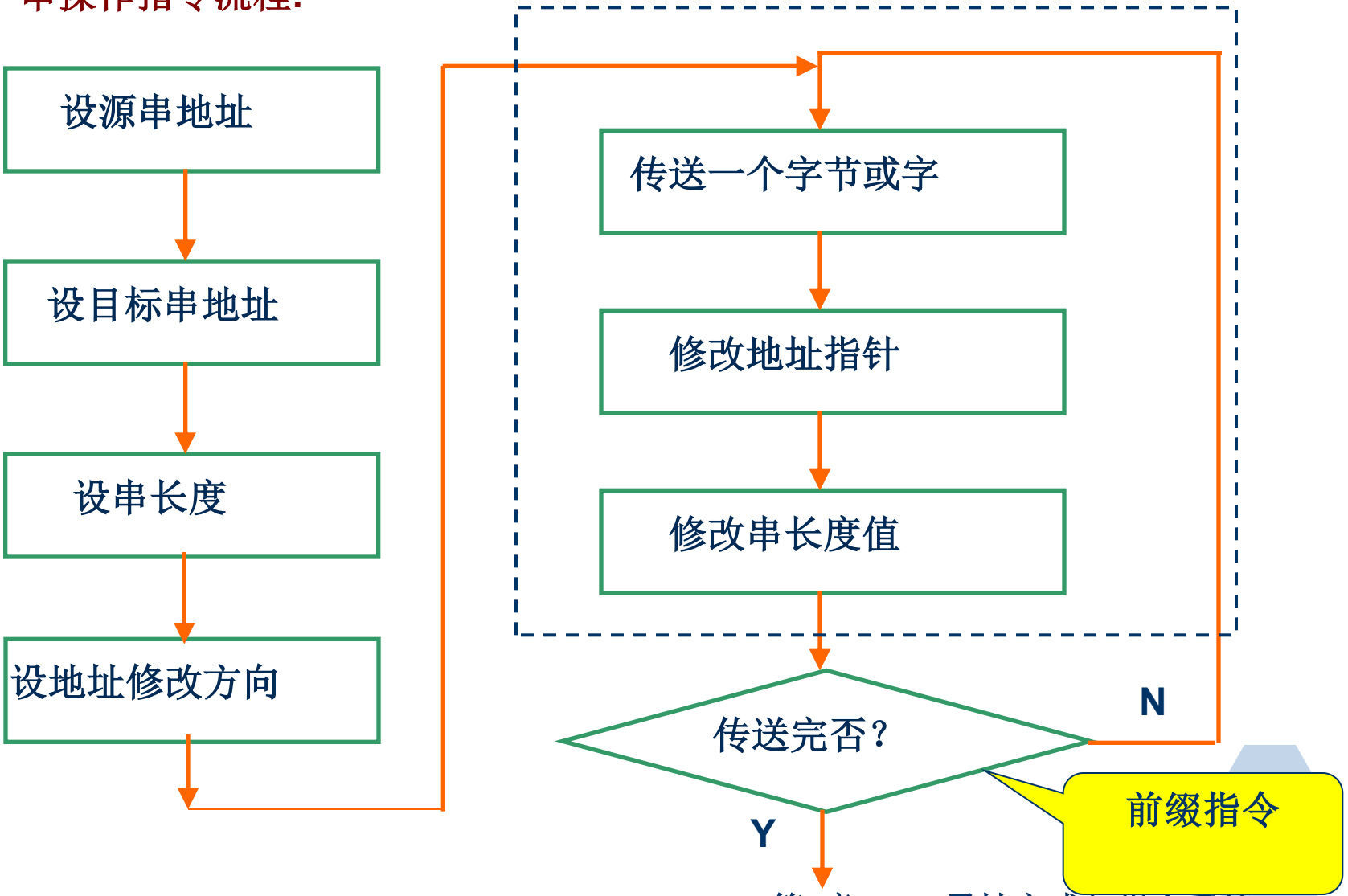
传送过程中，如方向标志DF 为0，则SI和DI 做自动增 量变化，即每传送1 个字节，自动加1，而每传送1 个 字，自动加2；如果DF为1，则SI 和DI 作自动减量变化，即每传送1 个字节，自动减1，而每传送1 个字， 自动减2。

串传送指令例

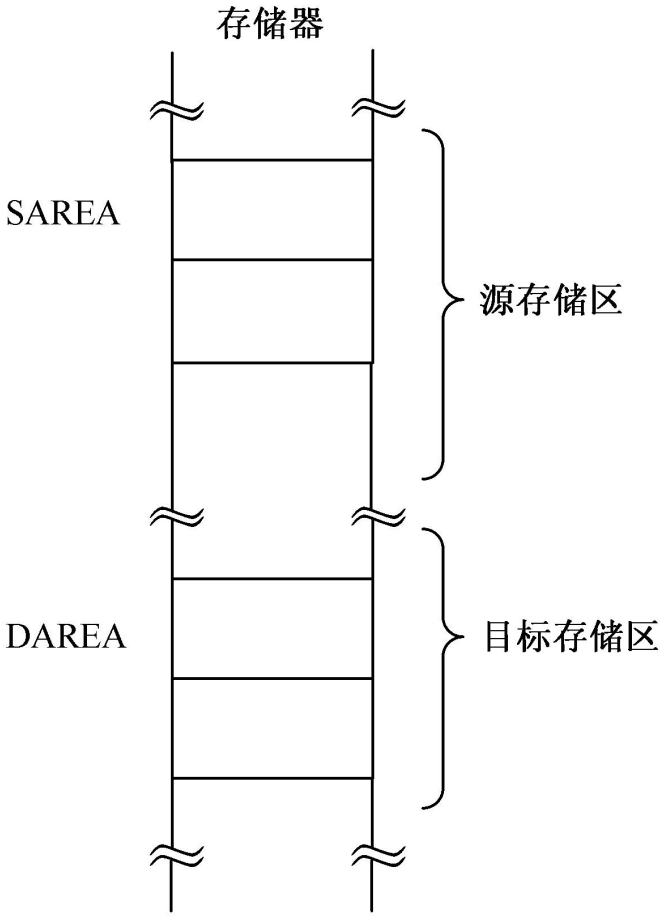
用串传送指令实现200个字节数据的传送：

```
LEA  SI，MEM1
LEA  DI，MEM2
MOV  CX，200
CLD
REP  MOVSB
HLT
```

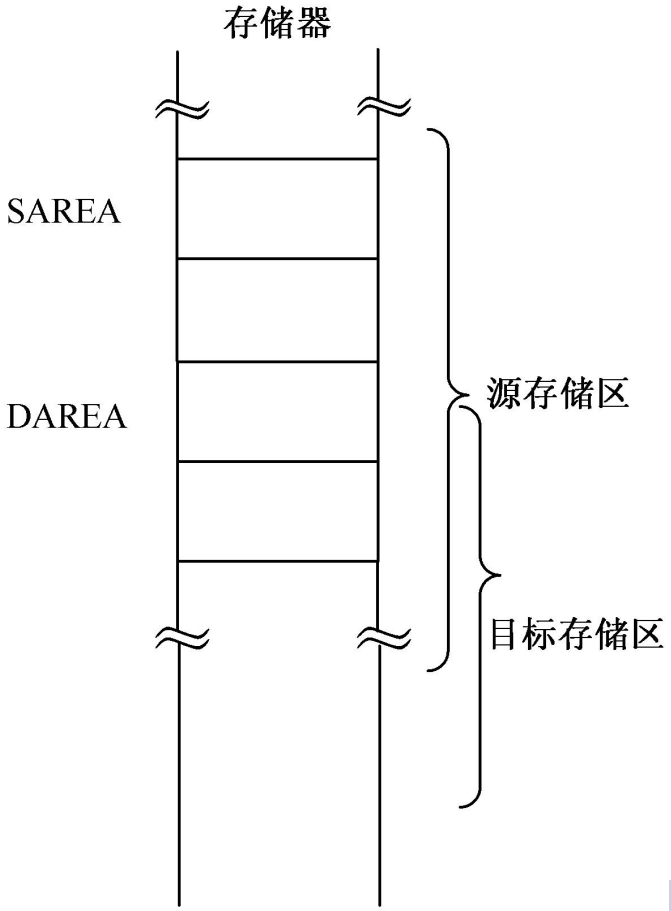
串操作指令流程:


设源串地址
设目标串地址
设串长度
设地址修改方向
传送一个字节或字
修改地址指针
修改串长度值
传送完否?
N
Y
前缀指令

【例4-18】 编写汇编程序段，把自SAREA开始的100个字复制到DAREA开始的区域中。



（a）源存储区与目标存储区不重叠



（b）源存储区与目标存储区重叠

重复前缀:

REP　无条件重复 → CX≠0 重复

REPE　相等重复
REPZ　为零重复
} CX≠0 或 ZF=1

REPNE　不相等重复
REPNZ　不为零重复
} CX≠0或 ZF=0

} 条件重复

CMPSB 和CMPSW 指令（Compare byte string ,compare word string）

- 功能：CMPSB /CMPSW 指令对存储器内的2 个字节/字串进行比较。
- 标志：AF、CF、OF、PF、SF、ZF 受影响
- 说明：串比较指令常与条件重复前缀连用，指令的执行不改变操作数，仅影响标志位。

串比较指令例

测试200个字节数据是否传送正确：

```
LEA  SI，MEM1
LEA  DI，MEM2
MOV  CX，200
CLD
REPE   CMPSB
TEST   CX，00FFH
JZ  STOP
DEC  SI
MOV  AL，[SI]
MOV  BX，SI
STOP：HLT
```

?

SCASB/SCASW 指令（Scan byte string or scan word string）

- 功能：按照累加器中给出的字节或字对ES：DI 所指的字节串或字串进行检索，每检索一次，便对DI 作一次修改。
- 标志：影响AF、CF、OF、PF、SF、ZF
- 说明：常用于在指定存储区域中寻找某个关键字。即。通过加重复前缀实现在字节串中或字串中检索和累加器内容相同的或不同的字节或字。

比如：

REPE SCASB 用来检索和AL 内容不同的字节，

REPNZ SCASW 用来检索和AX 内容相同的字。

LODSB/LODSW 指令（Load byte or word string）

- 功能：将源操作数提供的字节（字）取到AL（AX）。
- 标志：所有标志不受影响。
- 说明：每次重复执行指令会把累加器的内容冲掉，因此，这两条指令只用于循环程序中，而不加重复前缀。

STOSB/STOSW 指令（Store byte sring or store word string）

- 功能：将AL（AX）中的字节（或字）存入ES：DI 所指单元。
- 标志：所有标志不受影响。
- 说明：用于将输入的一串数据填入内存一个区域；STOSB和STOSW 指令前加重复前缀时，可以使内存一个区域填满某个值。

4.3.5 程序控制指令

- 转移指令
- 循环控制
- 过程调用
- 中断控制

1. 转移指令

通过修改指令的偏移地址或段地址及偏移地址实现程序的转移

- 无条件转移指令

  无条件转移到目标地址，执行新的指令

- 有条件转移指令

  在具备一定条件的情况下转移到目标地址

JMP 指令（Jump）—无条件转移指令

- 功能：IP←(IP)+以标号或立即数给出的偏移量，(CS)不变。
- 说明：可以为段内直接转移、段内间接转移、段间直接转移、段间间接转移。

段内转移:　　转移的目标地址在当前代码段内

段内直接寻址 → 指令中直接给出目标地址

例如:

```
JMP  2010H                ;IP←2010H
JMP  SHORT   L            ;IP←(IP)+8位移量
JMP  NEAR    NEXT         ;IP←(IP)+16位移量
```

段内间接寻址 → 由指令中的寄存器或存储器操作数指出目标地址

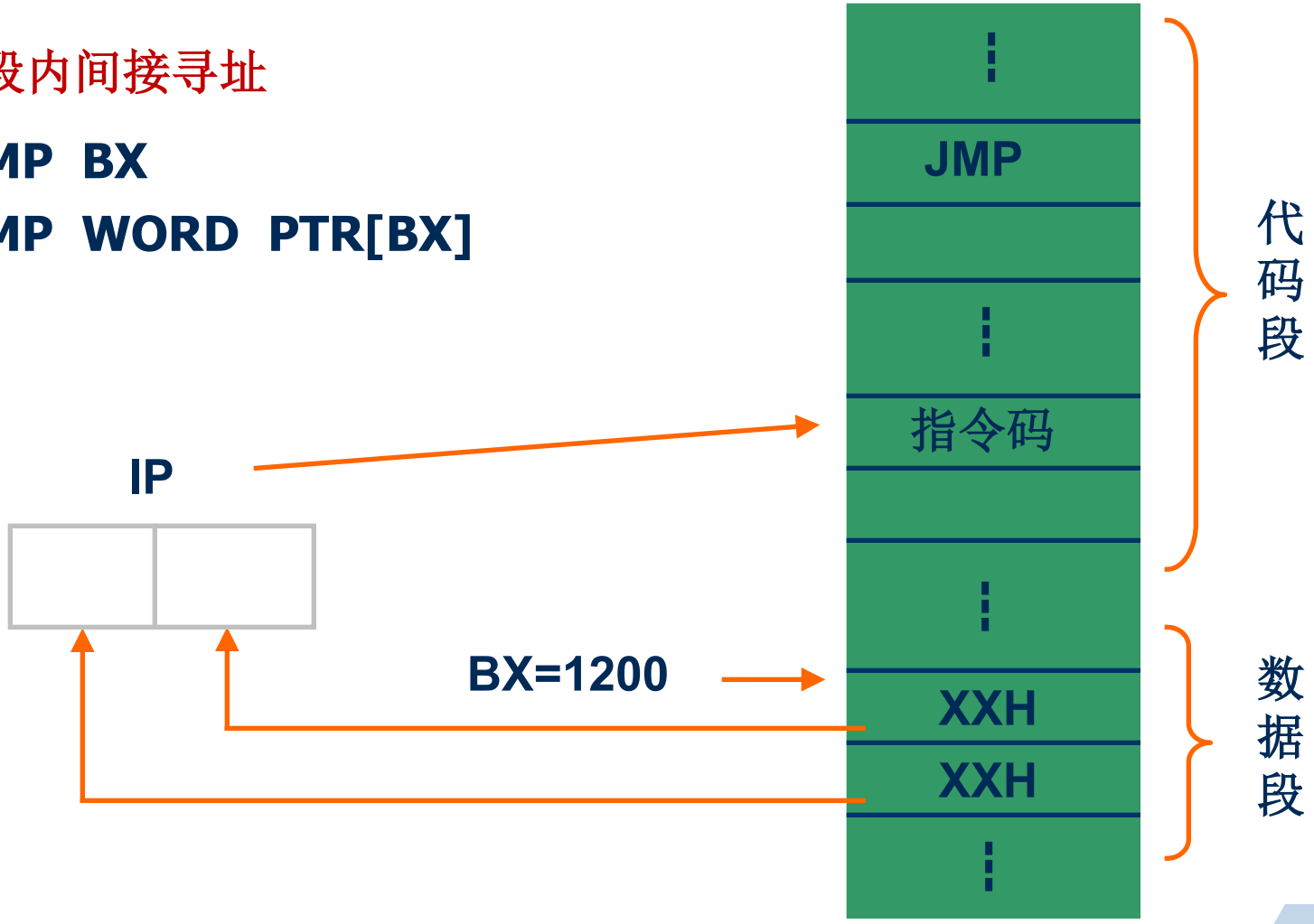
例如:

```
JMP  BX                   ;IP←(BX)
JMP  WORD PTR [SI]        ;IP←(DS:SI)
```

- 段内间接寻址

JMP BX

JMP WORD PTR[BX]



段间转移

- 段间直接寻址 → 指令中直接给出目标地址
- 段间间接寻址 → 由指令中的32位存储器操作数指出目标地址

转移的目标地址不在当前代码段内

例如：

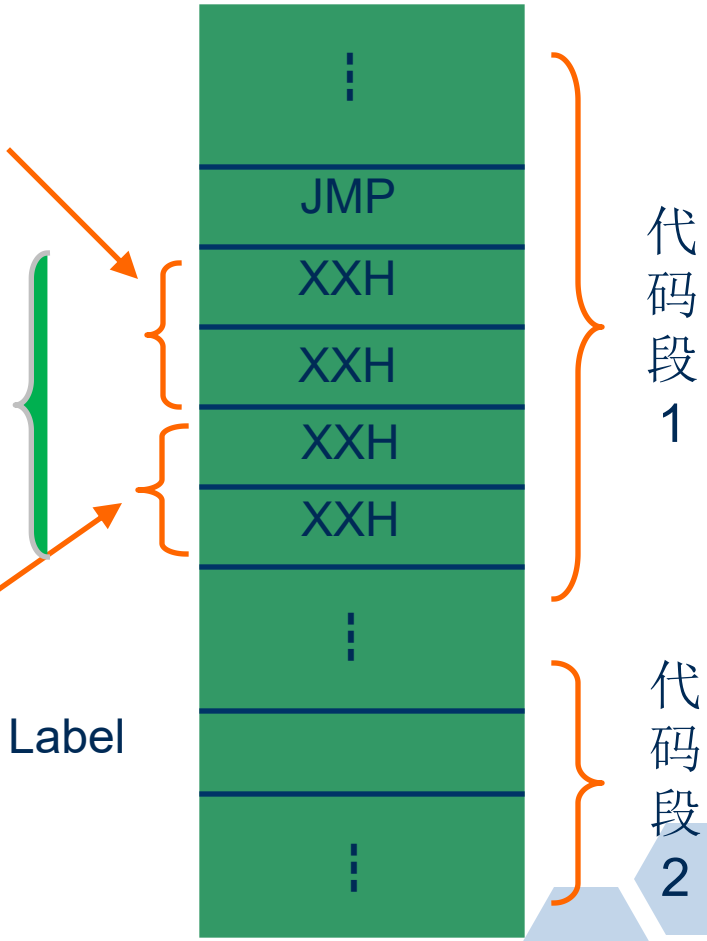
```
JMP  FAR  PTR  NEXT
                    ;(IP)←标号NEXT的偏移地址
                    ;(CS)←标号NEXT的段地址
JMP  DWORD  PTR  [SI]
                    ;IP←(DS:SI),CS←(DS:SI+2)
```

- 段间直接寻址

JMP FAR Label

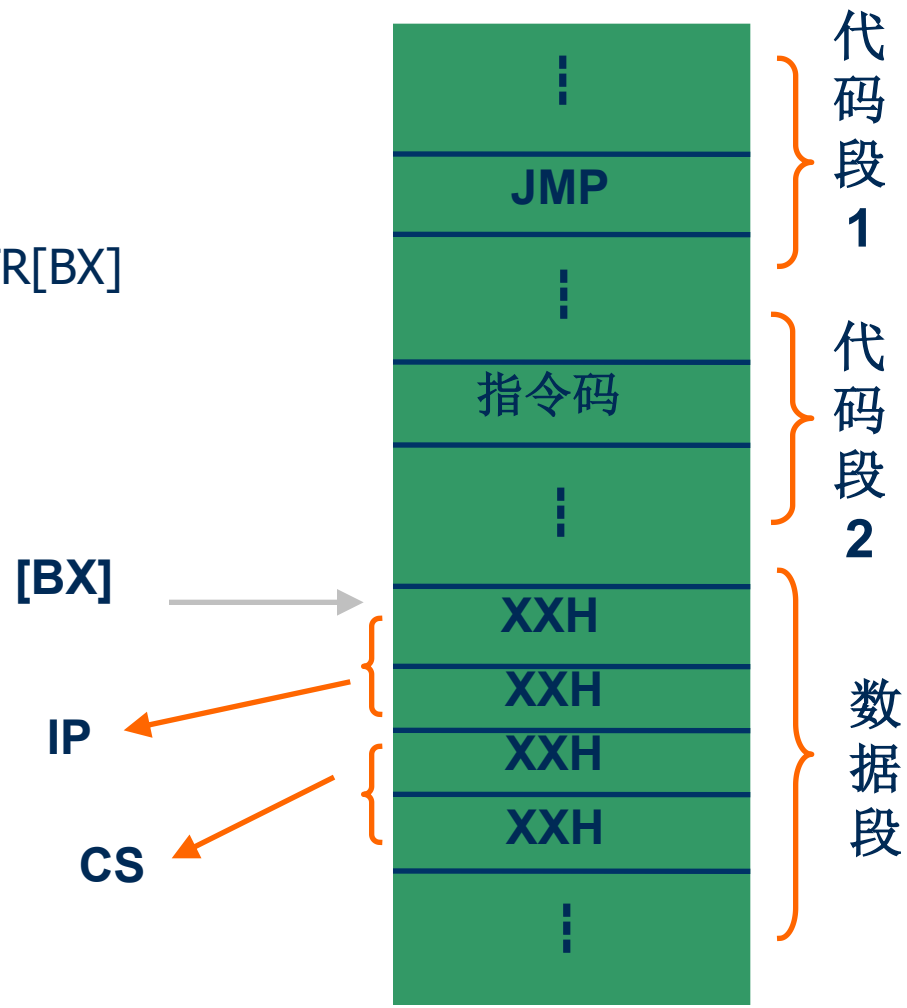
远地址标号

Label的段地址和段内偏移量



- 段间间接寻址

JMP DWORD PTR[BX]



条件转移指令

- 在满足一定条件下，程序转移到目标地址继续执行
- 条件转移指令均为段内短转移，即转移范围为：

  -128～+127

（1）根据单个标志位的状态判断的转移指令

指　令	转移条件	说　明
JC dest	CF=1	有进位/借位时，转移
JNC dest	CF=0	无进位/借位时，转移
JZ/JE dest	ZF=1	相等或等于0时，转移
JNZ/JNE dest	ZF=0	不相等或等于0时，转移
JS dest	SF=1	为负数时，转移
JNS dest	SF=0	为正数时，转移
JO dest	OF=1	有溢出时，转移
JNO dest	OF=0	无溢出时，转移
JP dest	PF=1	1的个数为偶数时，转移
JNP dest	PF=0	1的个数为奇数时，转移

（2）根据两个无符号数的比较结果判断的转移指令

指　令	转移条件	说　明
JA/JNBE dest	CF＝0且ZF＝0	X>Y时，转移
JAE/JNB dest	CF＝0或ZF＝1	X≥Y时，转移
JB/JNAE dest	CF＝1且ZF＝0	X<Y时，转移
JBE/JNA dest	CF＝1或ZF＝1	X≤Y时，转移

2. 循环控制指令

- 控制程序在以当前IP为中心的-128～+127范围内循环执行
- 循环次数由CX寄存器指定

循环指令	LOOP	CX ≠ 0
相等循环指令	LOOPZ	CX ≠ 0 或 ZF=1
不相等循环指令	LOOPNZ	CX ≠ 0 或 ZF=0

⑩ 【例4-21】 在首地址为Array的存储区域已存入长度为M的字数组，编写汇编程序段，统计该数组中0元素的个数，统计结果存入Result单元。

解：

```
        MOV     CX,M
        MOV     Result,0
        MOV     SI,0
AGAIN:  MOV     AX,Array[SI]
        CMP     AX,0
        JNZ     NEXT
        INC     Result
NEXT:   ADD     SI,2
        LOOP    AGAIN                   ;进入下一轮循环
        HLT
```

说明：LOOP AGAIN等效于下列语句：

```
DEC  CX
JNZ  AGAIN
```

LOOP指令中完成CX←CX-1的操作不影响标志位，DEC指令对标志位会影响。

3. 过程调用和返回

调用指令的执行过程

- 将调用指令的下一条指令的地址（断点）压入堆栈
- 获取子过程的入口地址（子过程第1条指令的偏移地址）
- （执行子过程，含相应参数的保存及恢复）
- 将断点偏移地址由堆栈弹出，返回原程序

- 段内调用
  - 段内直接调用
  - 段内间接调用
- 段间调用
  - 段间直接调用
  - 段间间接调用

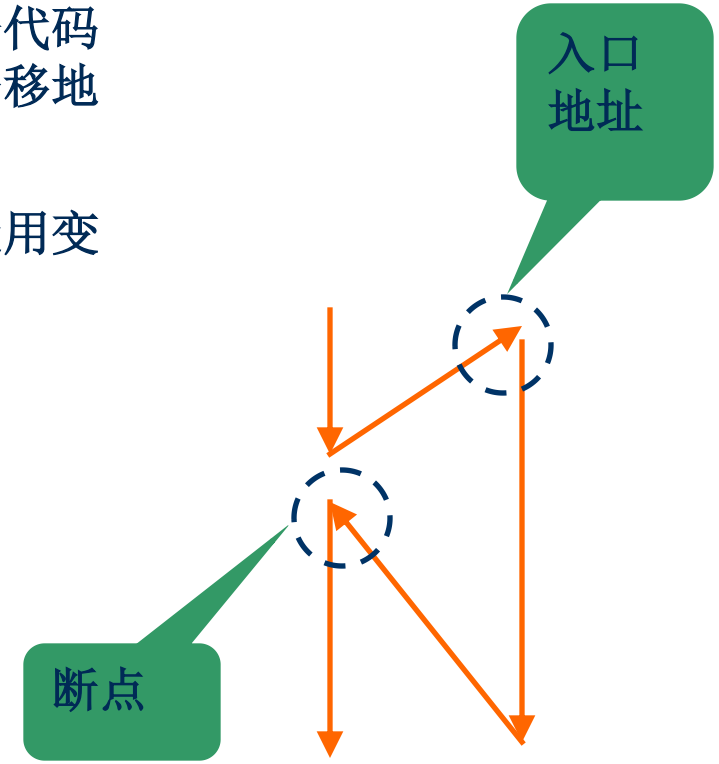
段内调用

- 说明：子过程与原调用程序在同一代码段，在调用之前只需保护断点的偏移地址(IP)。
- 约束：直接调用时，转移地址不能用变量，而只能用相对于IP 的标号。

例：CALL ABC

CALL WORD [BX+SI]

CALL BX



段间调用

说明：子过程与原调用程序不在同一代码段，在调用之前需保护断点的段基地址和偏移地址。先将断点的**CS**压栈，再压入**IP**

例：**CALL FAR TIMRE**

CALL DWORD PTR [BX]

返回指令

操作：从堆栈中弹出断点地址，返回原程序。

说明：RET指令一般位于子程序的最后。

注意， **RET** 指令的类型和过程调用指令 **CALL**的类型(段内返回类型/段间返回类型)要一致。

例　　RET

4. 中断调用和返回指令

（见中断控制器）

4.3.6 处理器控制指令

- 对标志位的操作
- 与外部设备的同步

例：

```
CLC
STI
HLT
NOP
```

标志位操作指令

指令格式	操　作	说　明
STC	CF←1	进位标志置1
CLC	CF←0	进位标志清0
CMC	CF←CF	进位标志取反
STD	DF←1	方向标志置1
CLD	DF←0	方向标志清0
STI	IF←1	中断允许标志置1，开中断
CLI	IF←0	中断允许标志清0，关中断

思考题：1、2、3、12、13、15

习题：7、8、9、10、11、14、16、18、20、21、27